НП ”Обучение за ИТ умения и кариера”

Модул 7: Разработка на софтуер

КУРСОВ ПРОЕКТ

на тема:

Куриерска фирма

Изготвили:

Стоян Стойков

Мерт Юсуф

Група 07

гр. Хасково

2025 г.

I. Цели

Разработване на мулти-интерфейсно приложение, което управлява база данни, съдържаща информацията за пратки, клиенти, куриери и допълнителни услуги.

II. Разпределение на ролите

Стоян Стойков-Отговаря за базата данни (Models и DbContext) , тестовете и за малка част от Business логиката.Мерт Юсуф-Отговаря за изгледа на проекта (Views) , по-голямата част от Business логиката на проекта и ASP.NET часта .Структурата на проекта и документацията са екипно изпълнени.

III. Основни етапи в реализирането на проекта

1. Планиране на структура

   Планиране на структурата на базата данни и разпределението на структурата на проекта. Избор на помощни библиотеки за разработка и избиране на GitHub за контролер на версиите.

2. Разработка на Data слоя

   Добавихме Entity Framework Core и допълнителни библиотеки нужни за разработката към проекта Data. Създадохме DbContext в класът ExpressDbContext и модели на таблиците в Data/Models. Описахме connection string и добавихме опция за инжектиране на connection string от ASP.NET. Описахме връзките на моделите за по-точно създаване на базата данни. Добавяме миграции и обновяваме базата данни чрез PM конзолата.

3. Разработка на Business слоя

   Добавихме нужните библиотеки и референция към проекта Data. За всеки модел създадох business клас, в който се описват CRUD операциите към моделите. И добавихме Dispose метод от IDisposable интерфейса за премахване на контекста.

4. Разработка на конзолно приложение

Добавихме референции към Business и Data слоя, както и нужните библиотеки за всички приложения. Разработихме Presentation слой, който ще управлява интерфейса на конзолното приложение. Той се състои от един главен клас Display, класове Sub-displays и един помощен клас MiniDisplay. При разработката на слоя установихме, че ще е много обемен ако цялата логика е само в Display класа, затова го разделихме на Sub-displays, като всяка таблица има свой sub-display. При разработката също създадохме помощния клас MiniDisplay, който има за цел да помага с входа на данни и да намали обема на кода, като в него се съдържат валидациите за входните данни при въвеждане в интерфейса на конзолата. Display класът събира логиката на sub-display класовете, в тях се съдържат менюта и методите които извършват, те използват методите от Business слоя.

5. Разработка на Тестове за Business логиката

Създадохме тестове за да сме сигурен дали валидно се изпълняват методите в Business логиката на всяка таблица. При разработката пробвахме да използвам Moq библиотеката, но тя не подържа async методи затова смених на In Memory Database библиотеката към EF, която не само направи процеса по-лесен, а и намали обема код и ускори времето за изпълнение на тестовете.

6. Разработка на ASP.NET (MVC) уеб приложение

При създаването на ASP.NET уеб приложението реших сам да напиша controller и view логиката на всеки клас вместо да използвам scaffolding, за да може да имам повече контрол над методите на контролерите и дизайна на страниците. В дизайна също използвам JavaScript за някои елементи, като търсачки и филтри. Също и в ASP.NET приложението ми се съдържа и Seeding слоя, който има за цел, в тестова среда, когато базата данни е празна, да се попълни с тестови данни за по-лесно презентиране и тества на проекта.

IV. Реализация

Използвани технологии:

Език и платформи:

1. C#
2. JavaScript
3. ASP.NET Core MVC

ORM и база данни:

1. Microsoft.EntityFrameworkCore – ORM за работа с бази данни
2. Microsoft.EntityFrameworkCore.SqlServer – SQL Server провайдър
3. Microsoft.EntityFrameworkCore.Tools – инструменти за миграции и работа с базата
4. Microsoft.EntityFrameworkCore.InMemory – In-Memory база за тестване
5. Microsoft.VisualStudio.Web.CodeGeneration.EntityFrameworkCore – за scaffold и code generation

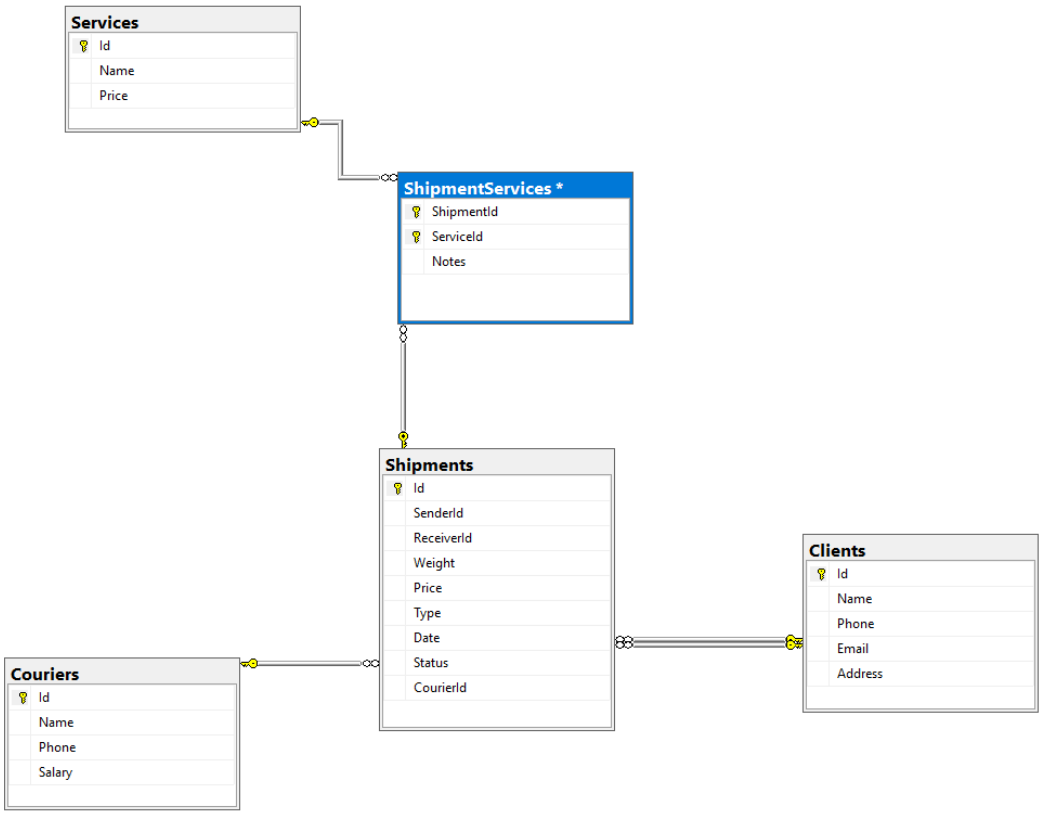
Тестване и покритие

1. XUnit – инфраструктура за тестове
2. coverlet.collector – за измерване на покритие на тестовете (code coverage)

Инструменти и други

1. GitHub – контрол на версиите
2. Visual Studio – среда за разработка
3. Package Manager Console – използвана за миграции и обновяване
4. Fine Code Coverage - Отбелязва покритието на unit тестове

V.ER диаграма на базата данни DeliveryTrackingDB



V. Връзки между таблиците

1. **Clients → Shipments (1:MANY)**
   Един клиент може да има множество пратки, но всяка пратка принадлежи на един конкретен клиент.
   Тази връзка се реализира чрез външния ключ **ReceiverId** (или SenderId, ако се използва и за изпращач) в таблицата **Shipments**.

---

2. **Couriers → Shipments (1:MANY)**
   Един куриер може да доставя множество пратки, но всяка пратка се обслужва от един куриер.
   Тази връзка се реализира чрез външния ключ **CourierId** в таблицата **Shipments**.

---

3. **Shipments → ShipmentServices (1:MANY)**
   Една пратка може да има множество допълнителни услуги, но всеки запис в ShipmentServices принадлежи на една пратка.
   Тази връзка се реализира чрез външния ключ **ShipmentId** в таблицата **ShipmentServices**.

---

4. **Services → ShipmentServices (1:MANY)**
   Една услуга може да бъде използвана в много пратки, но всеки запис в ShipmentServices се отнася за една конкретна услуга.
   Тази връзка се реализира чрез външния ключ **ServiceId** в таблицата **ShipmentServices**.

---

5. **Shipments ↔ Services (MANY:MANY)**
   Между таблиците **Shipments** и **Services** съществува много-към-много (M:N) връзка, тъй като:
   • една пратка може да включва много услуги
   • една услуга може да се използва в много пратки

Тази връзка е реализирана чрез междинната таблица **ShipmentServices**, която съдържа и допълнителна информация като:

- **Notes**

VI. Таблици

1. Таблица „Clients”

Таблицата съхранява информация за клиентите.

• **Id (INT)** – уникален идентификатор за всеки клиент, автоинкрементиращ. Не може да бъде null.
• **Name (VARCHAR)** – име на клиента. Не може да бъде null.
• **Phone (VARCHAR)** – телефонен номер.
• **Email (VARCHAR)** – имейл адрес.
• **Address (VARCHAR)** – адрес на клиента.

---

2. Таблица „Couriers”

Таблицата съхранява информация за куриерите.

• **Id (INT)** – уникален идентификатор. Не може да бъде null.
• **Name (VARCHAR)** – име на куриера. Не може да бъде null.
• **Phone (VARCHAR)** – телефонен номер.
• **Salary (DECIMAL)** – заплата на куриера.

---

3. Таблица „Services”

Таблицата съдържа допълнителни услуги към пратките.

• **Id (INT)** – уникален идентификатор. Не може да бъде null.
• **Name (VARCHAR)** – име на услугата (напр. „Експресна доставка“, „Застраховка“). Не може да бъде null.
• **Price (DECIMAL)** – базова цена на услугата. Не може да бъде null.

---

4. Таблица „Shipments”

Таблицата съхранява информация за пратките.

• **Id (INT)** – уникален идентификатор. Не може да бъде null.
• **SenderId (INT)** – идентификатор на изпращача.
• **ReceiverId (INT)** – идентификатор на получателя (връзка с Clients).
• **Weight (DECIMAL)** – тегло на пратката.
• **Price (DECIMAL)** – цена на доставката.
• **Type (VARCHAR)** – тип на пратката.
• **Date (DATETIME)** – дата на изпращане.
• **Status (VARCHAR)** – статус (напр. „В процес“, „Доставена“).
• **CourierId (INT)** – идентификатор на куриера (външен ключ към Couriers).

---

5. Таблица „ShipmentServices”

Междинна таблица, която свързва пратките и услугите.

• **ShipmentId (INT)** – идентификатор на пратката. Външен ключ към Shipments. Не може да бъде null.
• **ServiceId (INT)** – идентификатор на услугата. Външен ключ към Services. Не може да бъде null.

- **ExtraPrice (DECIMAL)** – допълнителна цена за услугата (ако има такава).
- **Notes (VARCHAR)** – допълнителни бележки.

Описание на приложението:

Разработеното приложение представлява библиотечна система за управление на книги, автори, читатели и история на заеманите книги. Системата използва многослойна архитектура, състояща се от:

- Data слой – съдържа модели на таблиците и контекста на базата данни чрез Entity Framework Core;
- Business слой – реализира бизнес логиката и CRUD операциите за всеки модел;
- Конзолен интерфейс – позволява базово управление чрез конзолно меню, разделено по функционалност за всяка таблица;
- ASP.NET Core MVC интерфейс – уеб базиран потребителски интерфейс, позволяващ визуално управление и търсене на записи;
- Тестов слой – включва автоматизирани тестове за валидация на бизнес логиката с помощта на In-Memory база и XUnit.

Приложението е подходящо за малки библиотеки или учебни цели. То позволява:

- Добавяне, редакция и изтриване на книги, автори и читатели;
- Проследяване на история на заетите книги;
- Филтриране и търсене по различни критерии;
- Автоматично зареждане на тестови данни в празна база (Seeding).

Покритие на тестовете върху business слоя

Business.BaseBusiness<T>	6	12	18	77	33.3%	0	2	0%	
Business.ClientBusiness	4	1	5	18	80%	0	0		
Business.CourierBusiness	16	1	17	40	94.1%	0	0		
Business.ServiceBusiness	1	1	2	15	50%	0	0		
Business.ShipmentBusiness	18	1	19	37	94.7%	0	0		
Business.ShipmentServiceBusiness	31	11	42	65	73.8%	4	6	66.6%	

Поради BaseBusiness класа има проблем с инструмента и не може да разчете над 50% на класа ServiceBusiness

V. Развитие и нововъведения

Към проекта може да се добави Windows Form интерфейс, както и да се надгради с профили в системата, като има различни нива потребители. Така приложенията ще могат да се ползват и от потребителите онлайн за да заемат книги, проверяват наличности и други данни от дома си.

Заключение

В хода на разработката на проекта успяхме да приложим на практика знанията си от Модул 7, свързани с разработката на многослойни приложения, бази данни и уеб технологии. Чрез реализирането на библиотечната система изградихме пълно функционално приложение с няколко интерфейса (конзолен и уеб), валидирана бизнес логика и автоматизирани тестове. Работата по проекта ни помогна да усъвършенстваме уменията си в работата с Entity Framework Core, ASP.NET MVC, тестове с XUnit, както и в изграждането на добре структурирана и мащабируема архитектура. Смятаме, че проектът изпълнява поставените цели и демонстрира ключови практики в реалната софтуерна разработка.